

STM32F446RE

Air-Quality-Driven AC Motor (Fan) Controller

System Design & STM32CubeMX Configuration Reference

Sensors: SPS30 • BME680/688 • MQ-7 • SCD30 | Actuator: RobotDyn AC Dimmer (TRIAC) | HAL Driver Layer

Using `delay()` inside the ISR (common in Arduino examples) causes long blocking interrupts at low power levels. Using a hardware timer (TIM one-pulse mode) instead keeps the zero-cross interrupt short and non-blocking — this is the approach used in this design.

⚠ Motor caveat: a fan motor is an inductive load, unlike a resistive lamp. Simple phase-cut dimming can cause audible hum/buzz and reduced torque at low speed, and may stall the motor below a usable threshold. Test the actual fan's usable dimming range — many AC fan motors only dim smoothly from roughly 40-100%. Below that, cycle-skipping (burst-fire) control may be needed instead of phase-cut.

4. Peripheral & Pin Allocation (Nucleo-F446RE)

Function	Peripheral	Pin(s)
SCD30	I2C1	PB8 (SCL), PB9 (SDA)
SPS30 + BME680	I2C2	PB10 (SCL), PB3 (SDA)
MQ-7 analog read	ADC1, IN0	PA0
MQ-7 heater switch	GPIO Output	PA1
RobotDyn ZC (zero-cross in)	GPIO EXTI, falling edge	PA10
RobotDyn PSM (TRIAC gate out)	GPIO Output (driven by TIM ISR)	PA6
Phase-delay timing	TIM2, one-pulse mode	internal
Debug / telemetry	USART2	PA2 (TX), PA3 (RX) — ST-LINK VCP
Status LED	GPIO Output	PA5 (onboard LED)

RobotDyn logic side accepts 3.3V or 5V — connects directly to the Nucleo's 3.3V logic with no level shifting needed. Only the logic-side GND should be common with the STM32; never connect the mains-side of the module to STM32 ground or any other circuitry.

5. STM32CubeMX Configuration

5.1 Clock Configuration

- HSE from ST-LINK, 8 MHz → PLL → SYSCLK = 180 MHz
- APB1 = 45 MHz, APB2 = 90 MHz (standard Nucleo-F446RE max configuration)

5.2 I2C1 — SCD30 (dedicated, slow bus)

- Mode: I2C
- Speed: Standard Mode, 100 kHz (SCD30 maximum per datasheet)
- Increase HAL I2C timeout to accommodate clock-stretch up to ~150 ms
- NVIC: enable I2C1 event + error interrupts if using IT/DMA mode (recommended)

5.3 I2C2 — SPS30 + BME680

- Mode: I2C
- Speed: 100 kHz (SPS30 constraint)

- Standard or DMA mode is fine

5.4 ADC1 — MQ-7

- Channel IN0 (PA0), 12-bit resolution
- Single channel, software or timer-triggered
- F446 has no hardware oversampler — average ~16 samples in software

5.5 GPIO

- PA1 — Output, push-pull (MQ-7 heater MOSFET switch)
- PA6 — Output, push-pull, low speed (RobotDyn PSM / gate drive, toggled from TIM2 ISR — not configured as a PWM channel)
- PA10 — GPIO_EXTI10, falling edge, pull-up enabled (RobotDyn ZC input — confirm actual polarity on a scope)
- PA5 — Output (status LED)

5.6 EXTI — Zero-Cross Input

- Configure the ZC pin's EXTI line, enable NVIC interrupt
- Preemption priority: 0 (highest in the system)
- HAL_GPIO_EXTI_Callback() computes the phase delay and arms TIM2 one-pulse mode

5.7 TIM2 — Phase Delay (One-Pulse Mode)

- Mode: One Pulse Mode
- Prescaler: set for ~10 μ s/tick (e.g. PSC = 449 on a 45 MHz APB1 timer clock → 100 kHz tick rate)
- ARR: computed at runtime from desired power % (0 to ~90-95% of the 10 ms half-cycle window)
- Enable Update interrupt — fires the PSM gate pulse once, then auto-stops
- NVIC preemption priority: 1 (second-highest)

5.8 USART2 — Debug / Telemetry

- 115200 baud, 8N1, via ST-LINK VCP (PA2/PA3) for logging sensor values during tuning

5.9 NVIC Priority Summary

Interrupt Source	Priority	Rationale
EXTI (zero-cross)	0 (highest)	Must never be delayed — timing anchor for all TRIAC firing
TIM2 (gate pulse timing)	1	Must fire precisely relative to the zero-cross event
I2C1 / I2C2 / ADC / USART	3-5 (lower)	Sensor polling jitter must never block dimmer timing

6. Software Architecture

```
main.c
├── Init all HAL peripherals (CubeMX generated)
├── main loop (or FreeRTOS – recommended)
│   ├── Task: Sensor polling (I2C1 / I2C2 / ADC) – every 1-2 s, non-blocking / DMA
│   └── Task: Air-quality → fan-speed algorithm
```

```

└─ Task: Telemetry / logging via UART
└─ ISR: EXTI zero-cross      → arm TIM2 one-pulse mode
└─ ISR: TIM2 update          → fire TRIAC gate (PSM pulse)
└─ ISR / state machine: MQ-7 heater duty cycle

```

Given the mix of I2C/analog sensors with differing timing needs (SCD30 measurement cadence, MQ-7 heater cycling every 60-90 s, BME680 gas-heating cycles) alongside a hard real-time TRIAC-firing requirement, FreeRTOS (selectable directly in CubeMX) is recommended over a bare superloop. It isolates sensor-polling jitter from the zero-cross ISR / timer logic.

7. Reference Dimmer Driver Code (HAL)

The following illustrates the zero-cross → phase-delay → gate-fire logic. For production use, replace the busy-wait pulse width with a second timer (e.g. TIM3) in one-pulse mode to avoid any blocking inside an ISR.

```

// Global state
volatile uint32_t phase_delay_ticks = 0; // 0 (full power) .. MAX (min power)
volatile uint8_t fan_power_percent = 0; // 0-100, set by control algorithm

#define MAINS_HALF_CYCLE_US 10000 // 50Hz -> 10ms; use 8333 for 60Hz
#define TIM2_TICK_US 10 // PSC configured so 1 tick = 10us
#define MAX_DELAY_TICKS ((MAINS_HALF_CYCLE_US / TIM2_TICK_US) - SAFETY_MARGIN)

// EXTI callback - fires on every zero cross
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_10)
    {
        phase_delay_ticks = MAX_DELAY_TICKS -
            ((fan_power_percent * MAX_DELAY_TICKS) / 100);

        __HAL_TIM_SET_AUTORELOAD(&htim2, phase_delay_ticks);
        __HAL_TIM_SET_COUNTER(&htim2, 0);
        HAL_TIM_Base_Start_IT(&htim2); // one-pulse mode: auto-stops after firing
    }
}

// TIM2 update callback - fires once, at the calculated delay after ZC
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_SET); // fire PSM
        // ~50-100us pulse width - use a second one-pulse timer in production
        for (volatile int i = 0; i < 400; i++) __NOP();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
    }
}

```

8. Suggested Next Steps

- Verify actual RobotDyn ZC signal polarity and idle state with an oscilloscope before finalizing EXTI edge configuration.
- Implement the non-blocking gate-pulse version using a second timer (TIM3) for pulse width.
- Develop sensor driver skeletons for SPS30, BME680, SCD30, and MQ-7 under HAL.
- Characterize the target fan motor's usable dimming range (phase-cut vs. cycle-skipping) before finalizing the control algorithm.
- Decide on bare-metal superloop vs. FreeRTOS based on final feature scope.